

# System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

## Complete Human Knowledge Transfer Guide

A senior engineer's brain-dump: mental models, gotchas and onboarding plan

|              |                                                          |
|--------------|----------------------------------------------------------|
| Project      | System Scanner Pro (v3.1)                                |
| Document     | Complete Human Knowledge Transfer Guide                  |
| Package date | August 21, 2026                                          |
| Prepared by  | Project Team - Kailasanathan (Team Leader), Saagr, Mega  |
| Audience     | Developers, administrators, testers and new team members |
| Status       | Final                                                    |

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

# Table of Contents

|                                                                          |   |
|--------------------------------------------------------------------------|---|
| 1. Why This Volume Exists . . . . .                                      | 3 |
| 2. The 60-Second Mental Model . . . . .                                  | 3 |
| 3. Ten Facts That Explain 80% of the Codebase . . . . .                  | 3 |
| 4. Repository Tour - The Route a Senior Would Walk You Through . . . . . | 3 |
| 5. Day-One Setup That Actually Works . . . . .                           | 4 |
| 6. Vocabulary You Will Hear in Standups . . . . .                        | 4 |
| 7. Gotchas That Cost Us Hours (Learn Them Cheap) . . . . .               | 4 |
| 8. How to Debug Like a Pro Here (Decision Tree) . . . . .                | 4 |
| 9. Reading List Order for New Developers . . . . .                       | 5 |
| 10. How to Make Your First Safe Contribution . . . . .                   | 5 |
| 11. Where the Bodies Are Buried (Honest Debt Map) . . . . .              | 5 |
| 12. Performance Intuition . . . . .                                      | 5 |
| 13. Security Instincts for This Codebase . . . . .                       | 5 |
| 14. Working With the Docs Package Itself . . . . .                       | 6 |
| 15. Seven-Day Onboarding Plan . . . . .                                  | 6 |
| 16. FAQ From Every New Joiner So Far . . . . .                           | 6 |
| 17. Code Review Checklist Specifics . . . . .                            | 6 |
| 18. When You Are On Call - Quick Cards . . . . .                         | 6 |
| 19. Culture Notes . . . . .                                              | 7 |

## 1. Why This Volume Exists

Volumes 01-08 document the system. This one transfers judgment - the things a senior developer would tell you across a desk during your first weeks: how to think about the codebase, where beginners stumble, which decisions are load-bearing, and how to become productive in seven days. Written in a direct voice on purpose.

## 2. The 60-Second Mental Model

Say this to yourself every time you get lost: **agents post facts; the server judges them; browsers watch.** Agents (client/) never touch business logic - they only collect and report. The Django server (admin/) owns truth: identity, approval, storage, scoring. Browsers are thin viewers over that truth plus a WebSocket tap for liveness. If a bug breaks this direction of trust, you have found an architecture bug, not just a code bug.

## 3. Ten Facts That Explain 80% of the Codebase

1. One Client row per machine; its registration\_key is the join key everywhere.
2. Approval is state, not ceremony - booleans approved/status + fingerprint history decide everything (read tests/test\_registration\_approval.py like scripture).
3. Scans are stored twice: raw JSON (ScanResult.scan\_data) for fidelity, normalised inventory rows for speed.
4. Heartbeats (30 s) drive liveness; two scheduler jobs flip stale devices to offline.
5. The WebSocket layer is optional sugar - REST works without it; Vercel mode proves it.
6. HMAC agent auth = shared secret per device (AgentSecret) + timestamp freshness window.
7. Company scoping is enforced in views via get\_user\_company(), not in the ORM by default - new views MUST scope manually.
8. Everything user-visible about alerts flows through intelligence.Alert; monitoring.DeviceAlert is the upstream feed.
9. Reports are code-generated documents (ReportLab/openpyxl) - templates live in Python builders, not HTML.
10. Deployment mode changes session mechanics: serverless forces signed cookies; local uses DB sessions unless told otherwise.

## 4. Repository Tour - The Route a Senior Would Walk You Through

| Stop | File                           | What to notice                                         |
|------|--------------------------------|--------------------------------------------------------|
| 1    | admin/main.py                  | How much orchestration one honest entry point can hold |
| 2    | admin/django_admin/settings.py | Branching by environment; the pooler rewrite trick     |
| 3    | scanner_api/views.py           | Cluster order mirrors URL order; helpers at top        |
| 4    | scanner_api/tests/             | Executable specification of enrollment                 |
| 5    | monitoring/consumers.py        | Handshake-before-accept pattern                        |
| 6    | monitoring/scheduler.py        | Jobs are thin wrappers over querysets                  |
| 7    | intelligence/reports.py        | Builder-per-report-type convention                     |
| 8    | client/main.py                 | Thread topology: heartbeat, WS, monitors, dispatcher   |
| 9    | client/events/dispatcher.py    | Batch + disk-queue + replay pattern                    |
| 10   | deploy/install.sh              | Idempotency mindset for ops scripts                    |

Table 1 - Guided tour

## 5. Day-One Setup That Actually Works

```
git clone https://github.com/shinecraftinternship-kms/AdminClient.git
cd AdminClient
python -m venv .venv && .venv\Scripts\activate          # POSIX: source .venv/bin/activate
pip install -r requirements.txt
# admin/.env with DJANGO_SECRET_KEY set (see vol 03)
python admin/manage.py migrate
python admin/main.py --port 8000
```

Then in a second terminal: `python client/main.py "http://localhost:8000/connect/"` - approve the pending card and you have the full loop running in under ten minutes.

## 6. Vocabulary You Will Hear in Standups

| Term                    | Translation                                                         |
|-------------------------|---------------------------------------------------------------------|
| Pending card            | Unapproved Client awaiting review on the dashboard                  |
| Demotion                | Automatic loss of approval after identity/policy violation          |
| Diff                    | Computed delta between consecutive scans                            |
| Stale                   | Heartbeat older than threshold but not yet flipped offline          |
| scan_now                | Push command telling an agent to scan immediately                   |
| Self-client / ADMIN row | The server monitoring itself for demo/health                        |
| Pooler                  | Supabase transaction-mode Postgres port 6543 required on serverless |
| Signed cookie path      | serverless-friendly session fallback carrying signed user id        |
| Registry row            | server_registry table entry agents use for discovery                |
| Rescue mode             | Agent flag dumping diagnostics when things break                    |

Table 2 - Team vocabulary

## 7. Gotchas That Cost Us Hours (Learn Them Cheap)

- Changing fingerprints intentionally demotes devices - if a VM clones a disk, expect re-approval; do not 'fix' this logic casually.
- Auto-approve is hardcoded off; the settings toggle cannot enable it - UX messaging must not promise otherwise (test 3.2 exists because of this).
- On serverless, InMemory channel layer means WS features silently vanish - that is expected there, not a bug hunt.
- SECRET\_KEY rotations log everyone out AND invalidate JWTs - schedule them.
- PowerShell-based collectors need the agent's bypass invocation; restrictive GPOs can break scanning without any Python error.
- UDP 45000 discovery dies at routed VLAN boundaries - fall back to registry/connect URL rather than debugging 'network ghosts'.
- Report timeouts on Vercel are the 30 s cap talking - move heavy packs to VPS mode or narrow ranges.

## 8. How to Debug Like a Pro Here (Decision Tree)

```
Symptom: device wrong/offline/missing?
|-> Is it Pending?           -> approval flow (tests explain rules)
|-> Approved but offline?    -> heartbeat path: agent logs -> firewall -> stale jobs
|-> Data weird but online?   -> inspect last ScanResult JSON vs inventory rows
|-> Live UI frozen?          -> WS handshake logs; nginx upgrade headers; asgi flag
|-> Everything broken?        -> recent deploy? SECRET_KEY changed? DB reachable?
```

## 9. Reading List Order for New Developers

1. README.md (20 minutes).
2. Volume 02 figures (30 minutes).
3. settings.py end-to-end (45 minutes) - it encodes most operational wisdom.
4. urls.py files top-to-bottom - the API surface becomes familiar fast.
5. views.py auth cluster then RegisterClientView/PingClientView/SubmitScanView.
6. monitoring consumers + scheduler.
7. client/main.py thread dance, then events/dispatcher.py.
8. One report builder end-to-end in intelligence/reports.py.

## 10. How to Make Your First Safe Contribution

1. Pick something observable: a report column, an alert message, an empty-state hint.
2. Add/adjust a test when touching registration semantics - the suite is your safety net.
3. Run `manage.py test scanner_api` before pushing.
4. Keep views company-scoped: copy the `get_user_company()` pattern from any sibling view.
5. Update `MANUAL_TESTING.xlsx` if behaviour changes user-visible steps.

## 11. Where the Bodies Are Buried (Honest Debt Map)

| Area          | Debt                  | Safe direction                                                                                          |
|---------------|-----------------------|---------------------------------------------------------------------------------------------------------|
| views.py size | ~4,100 lines monolith | Split per cluster; keep URLs stable                                                                     |
| Permissions   | Global default empty  | Set <code>DEFAULT_PERMISSION_CLASSES</code> then fix fallout                                            |
| CORS          | Allow-all             | Restrict list; verify dashboard still works                                                             |
| Channel layer | In-memory             | Redis swap behind same interface                                                                        |
| Email         | Absent                | Introduce <code>EMAIL_*</code> settings + <code>send_mail</code> in forgot/reset + notifications digest |
| MFA fields    | Unused                | Wire TOTP (pyotp) or remove fields                                                                      |

Table 3 - Debt register with directions

## 12. Performance Intuition

- SQLite happily serves ~dozens of active devices; beyond ~50 go Postgres.
- Scan payloads are the big rows - keep `scan_interval` sane for chatty fleets.
- Anomaly windows grow with history; `health_checker` cadence (300 s) is the throttle.
- Indexes exist on `scan(client,time)`-style access paths; add more only with EXPLAIN evidence.

## 13. Security Instincts for This Codebase

- Treat every new endpoint as public until scoped: company filter + permission class + auth mixin.
- Never log tokens/secrets - audit tables already capture enough context.
- Agents are semi-trusted: validate payload shapes defensively (submit-scan does).
- The committed `.env.template` credentials are burned - rotate first, argue later (vol 06 C1).

## 14. Working With the Docs Package Itself

This package was generated from repository inspection; regenerate/update volumes when architecture shifts. Keep [TO BE PROVIDED]/[PROVIDED SEPARATELY]/[TO BE TESTED] markers honest - they are the difference between documentation and fiction.

## 15. Seven-Day Onboarding Plan

| Day | Goal                                  | Proof of done                                   |
|-----|---------------------------------------|-------------------------------------------------|
| 1   | Local stack + guided tour (section 5) | Login works; one agent enrolled                 |
| 2   | Registration semantics deep dive      | Explain all 12 approval tests aloud             |
| 3   | Telemetry + scheduler trace           | Follow one heartbeat from socket to row         |
| 4   | Scan pipeline + diffing               | Produce a diff and interpret it correctly       |
| 5   | Alerts/reports surface                | Generate all four key reports (Module 11 cases) |
| 6   | Deployment rehearsal                  | install.sh staging up; TLS green                |
| 7   | Manual test batch 1 (Modules 1-3)     | Results recorded in workbook                    |

Table 4 - Onboarding sprint plan

## 16. FAQ From Every New Joiner So Far

| Question                                       | Answer                                                                                                   |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Why both ScanResult JSON and inventory tables? | Audits want originals; queries want structure - we pay storage for both.                                 |
| Can I skip approvals in dev?                   | Enroll via ADMIN self-client or temporarily seed approval in a fixture - do NOT flip auto-approve logic. |
| Where do notifications go?                     | Notification rows + bell icon; email is roadmap.                                                         |
| Is the AI real ML?                             | Statistical engines (zscore/IQR/regression) - explainable by design; feature_store readies future ML.    |
| Why port 45000 UDP?                            | LAN broadcast discovery for zero-config enrollment; harmless if blocked thanks to fallbacks.             |
| How do I rotate an agent secret?               | Monitoring UI/API regenerates AgentSecret; agent re-handshakes on next connect.                          |

Table 5 - FAQ

## 17. Code Review Checklist Specifics

1. Company scoping present on every queryset touching tenants?
2. New endpoints covered by at least one automated or manual case?
3. No secrets/logs of secrets added?
4. Client-side changes keep stdlib-only discipline (plus websockets/watchdog)?
5. Docs volumes touched if architecture/ops behaviour changed?

## 18. When You Are On Call - Quick Cards

| Page                 | First 3 moves                                                                     |
|----------------------|-----------------------------------------------------------------------------------|
| Fleet-wide offline   | Server up? /api/health/ -> scheduler alive (ADMIN row online?) -> provider status |
| WS dead              | asgi flag? nginx upgrade headers? recent cert/proxy change?                       |
| Report timeout storm | Vercel cap? narrow ranges; move to VPS; check dataset growth                      |
| Lockout flood        | LoginAttempt sources; reset window; enforce credential hygiene                    |

*Table 6 - On-call cheat cards*

## 19. Culture Notes

Small team, high trust: write tests as documentation, keep the honesty markers intact, prefer boring technology when it suffices, and remember the platform's whole point - someone's 2 a.m. gets easier because a disk-full alert fired on Tuesday afternoon.